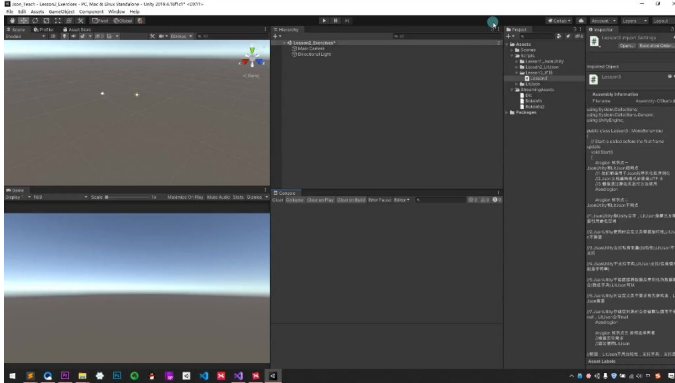


一、数据持久化 00:12

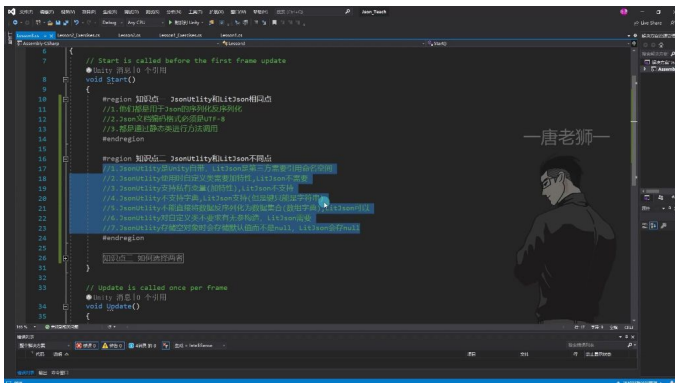
1. JsonUtility和LitJson对比 00:18

1) 相同点 00:41



-
- **核心功能：**两者都用于Json的序列化与反序列化，实现内存数据与Json字符串的相互转换
- **编码要求：**处理Json文档时必须使用UTF-8编码格式，否则反序列化会出现问题
- **调用方式：**均通过静态类提供方法调用（JsonUtility直接调用，LitJson需使用JsonMapper）

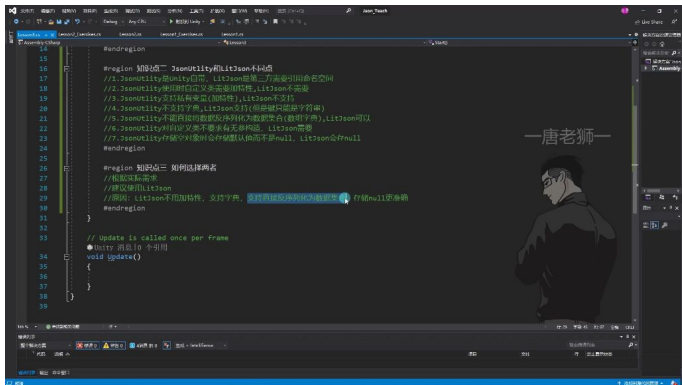
2) 不同点 01:55



-
- **来源差异：**
 - JsonUtility是Unity内置工具
 - LitJson是第三方库，需从GitHub下载并引用命名空间
- **特性要求：**
 - JsonUtility对嵌套自定义类需添加[Serializable]特性
 - LitJson无需任何特性标注
- **变量支持：**
 - JsonUtility支持私有变量（需添加[SerializeField]特性）
 - LitJson不支持私有变量序列化
- **字典处理：**
 - JsonUtility完全不支持字典类型
 - LitJson支持字典但键必须是字符串类型
- **集合处理：**
 - JsonUtility不能直接反序列化为数组/字典，需自定义包装类
 - LitJson可直接反序列化为List或Dictionary
- **构造要求：**

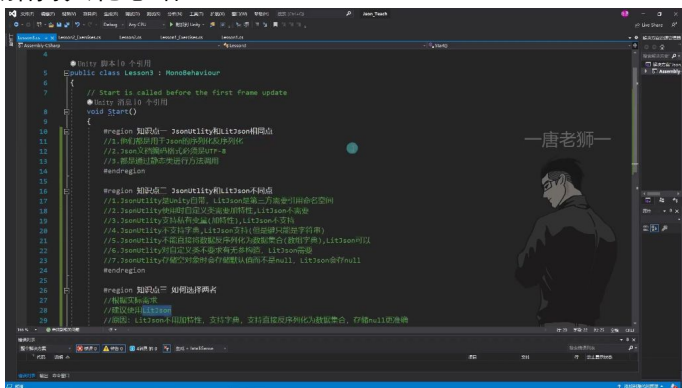
- JsonUtility对类无构造要求
- LitJson要求类必须有无参构造函数
- 空值处理：
 - JsonUtility存储空对象会转为默认值
 - LitJson会准确存储为null值

3) 如何选择 05:07



- 选择原则：根据实际项目需求决定
- 推荐方案：优先使用LitJson，原因包括：
 - 无需添加特性，编码更简洁
 - 完整支持字典类型操作
 - 可直接反序列化为集合类型
 - 空值处理更符合预期
 - 整体使用体验更接近标准Json处理流程

2. 数据持久化总结 06:10



- 知识体系：已完成Json持久化核心知识点的系统学习
- 实践建议：重点掌握LitJson的实际应用，其功能覆盖更全面
- 学习路径：后续将进入实践项目阶段巩固所学知识

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
JSON Utility vs. LitJSON 相同点	1. 均用于JSON序列化与反序列化 2. JSON文档编码格式必须为UTF-8 3. 通过静态类调用方法（JSON Utility直接调用，LitJSON需导入JsonMapper）	编码格式限制： 非UTF-8会导致解析失败	★★

JSON Utility vs. LitJSON 不同点	1. 来源：JSON Utility为Unity自带，LitJSON需第三方引用 2. 特性要求：JSON Utility需为自定义类添加[Serializable]特性，LitJSON无需 3. 私有变量支持：JSON Utility支持（需[SerializeField]），LitJSON不支持 4. 字典支持：JSON Utility不支持，LitJSON支持（键需为字符串） 5. 数据集合反序列化：JSON Utility需包裹自定义类，LitJSON直接支持数组/字典 6. 构造要求：JSON Utility无要求，LitJSON需无参构造 7. 空对象存储：JSON Utility存默认值，LitJSON存准确空值	关键差异： - 特性依赖 - 数据结构兼容性 - 反序列化灵活性	★★★
技术选型建议	推荐LitJSON： 1. 无需特性标注 2. 支持字典与直接数据集合反序列化 3. 空值存储更准确	适用场景： - 需高频处理复杂JSON结构 - 避免冗余代码	★★